



# Data analysis workflow in tidyr

## PROMPT STRUCTURE

I want you to act as a data scientist programming in R tidyr. You are given the {dataframe name} dataframe containing the columns {column name}. {Insert requirement}

### Example: Data Aggregation



I want you to act as a data scientist programming in R tidyr. You are given the 'employee' dataframe containing the columns employee\_id, name, department\_id, email, join\_date, current\_salary. Find the year with the most number of new joiners.



To find the year with the most number of new joiners, we need to extract the year from the `join\_date` column and then count the number of employees who joined in each year. Here's the code to do that:

```
bash Copy code

library(tidyr)

# assume 'employee' dataframe has already been loaded

# extract year from join_date column
employee$join_year <- format(as.Date(employee$join_date), "%Y")

# count number of employees who joined in each year
year_counts <- employee %>% group_by(join_year) %>% summarise(num_joiners =

# find the year with the most number of new joiners
max_year <- year_counts$join_year[which.max(year_counts$num_joiners)]
max_joiners <- max(year_counts$num_joiners)

cat("Year with the most number of new joiners:", max_year, "\n")
cat("Number of joiners in that year:", max_joiners, "\n")
```



# Data analysis workflow in tidyr

## PROMPT STRUCTURE

I want you to act as a data scientist programming in R tidyr. You are given the {dataframe 1 name} dataframe containing the columns {column name}. You also have a {dataframe 2 name} dataframe containing the columns {column name}. Find the {required output}

## Example: Data Merging



I want you to act as a data scientist programming in R tidyr. You are given the 'employee' dataframe containing the columns employee\_id, name, department\_id, email, join\_date, current\_salary. You also have a 'department' dataframe containing the columns department\_id and department\_name. Find the department\_name with the highest median salary.



To find the department\_name with the highest median salary, we need to perform a join operation between the 'employee' and 'department' dataframes based on the 'department\_id' column. Then, we can group the resulting dataframe by 'department\_name' and calculate the median salary for each group. Finally, we can sort the resulting dataframe by median salary in descending order and select the first row to get the department\_name with the highest median salary. Here's the code to do this:

```
bash Copy code

# Perform join operation on employee and department dataframes
joined_df <- merge(employee, department, by = "department_id")

# Group by department_name and calculate median salary
median_salary <- joined_df %>%
  group_by(department_name) %>%
  summarize(median_salary = median(current_salary))

# Sort by median salary in descending order and select top row
highest_median_salary <- median_salary %>%
  arrange(desc(median_salary)) %>%
  slice(1)

# Print department_name with highest median salary
cat("Department with highest median salary is", highest_median_salary$depart
```



# Data analysis workflow in tidyr

## PROMPT STRUCTURE

I want you to act as a data scientist programming in R tidyr. You are given the {dataframe name} dataframe containing the columns {column name}. Please convert the data to wide format.

### Example: Data Reshaping (Long to Wide)



I want you to act as a data scientist programming in R tidyr. You are given the `department_expense_2022` dataframe containing the columns `department_id`, `month`, `expense`. Please convert the data to a wide format.



To convert the `department_expense_2022` dataframe from long to wide format, we can use the `spread()` function from the tidyr package in R. We'll spread the data based on the `month` column and convert it to wide format using the `department_id` column as the id variable. Here's the code:



scss

Copy code

```
library(tidyr)

wide_expense <- spread(department_expense_2022, month, expense)
```

This will create a new dataframe `wide_expense` with the `department_id` column as the id variable and the `month` columns as new columns with their corresponding `expense` values.



# Data analysis workflow in tidyr

## PROMPT STRUCTURE

I want you to act as a data scientist programming in R tidyr. You are given the {dataframe name} dataframe containing the columns {column name}. Please convert the data to long format.

## Example: Data Reshaping (Wide to Long)



I want you to act as a data scientist programming in R tidyr. You are given the `department_expense_2022` dataframe with the columns `'department_id'`, `'Jan'`, `'Feb'`, ..., `'Dec'`. Please convert this data to a long format.



To convert the `'department_expense_2022'` data from wide to long format, we can use the `'tidyr'` package's `'gather()'` function.  

Here's the code to do that:

SCSS

 Copy code

```
library(tidyr)

department_expense_2022_long <- gather(department_expense_2022,
                                       key = "month",
                                       value = "expense",
                                       -department_id)
```

In this code, `'gather()'` function takes three arguments:

- `'department_expense_2022'`: the data frame to convert
- `'key'`: the name of the new column that will contain the month names
- `'value'`: the name of the new column that will contain the expense values
- `'-department_id'`: the column(s) to keep as id columns (in this case, `'department_id'` column)

The resulting `'department_expense_2022_long'` data frame will have three columns: `'department_id'`, `'month'`, and `'expense'`. The `'month'` column will contain the names of the original wide-format columns (e.g. "Jan", "Feb", etc.), and the `'expense'` column will contain the values from those columns.

# Prompts for data visualization workflows



# Creating plots in ggplot2

## PROMPT STRUCTURE

I want you to act as a data scientist coding in R. Given a dataframe {dataframe name} containing the columns {column names} Use ggplot2 to plot a {chart type and requirement}.



I want you to act as a data scientist coding in R. Given a dataframe 'gapminder' containing the column 'country', 'gdp', 'year'. Use ggplot2 to plot a line chart of GDP against the year for each country.



As a data scientist coding in R, here's how you can use ggplot2 to plot a line chart of GDP against the year for each country in a dataframe 'gapminder' containing the columns 'country', 'gdp', and 'year':

R

Copy code

```
# Load the ggplot2 package
library(ggplot2)

# Plot GDP against the year for each country
ggplot(gapminder, aes(x = year, y = gdp, group = country, color = country))
  geom_line() +
  labs(title = "GDP over Time by Country", x = "Year", y = "GDP") +
  theme_bw()
```

In this code, we first load the `ggplot2` package. We then use `ggplot()` to create a plot and specify the data frame `gapminder` and the x and y variables `year` and `gdp`. We also specify the `group` and `color` aesthetics to group the data by country and color the lines by country.

We add `geom_line()` to draw the lines for each country, and `labs()` to set the title, x-axis label, and y-axis label. Finally, we use `theme_bw()` to set the plot theme to black and white.

This code will produce a line chart of GDP against the year for each country in the 'gapminder' dataframe. Each country will be represented by a different color line.



# Gridplot visualizations in ggplot2

## PROMPT STRUCTURE

I want you to act as a data scientist coding in R. Given a dataframe {dataframe name} containing the columns {column names}. Use ggplot2 to plot a pair plot that shows the relationship of one variable against another.



I want you to act as a data scientist coding in R. Given a dataframe 'iris' containing the columns 'sepal\_length', 'sepal\_width', 'petal\_width', 'petal\_length'. Use ggplot2 to plot a pair plot that shows the relationship of one variable against another.



As a data scientist coding in R, here's how you can use ggplot2 to create a pair plot that shows the relationship of one variable against another in a dataframe 'iris' containing the columns 'sepal\_length', 'sepal\_width', 'petal\_width', and 'petal\_length':

R

Copy code

```
# Load the ggplot2 and ggally packages
library(ggplot2)
library(ggally)

# Create a pair plot
ggpairs(iris[,c("sepal_length", "sepal_width", "petal_length", "petal_width")])
```

In this code, we first load the `ggplot2` and `ggally` packages. We then use `ggpairs()` to create a pair plot that shows the relationship of one variable against another for the columns "sepal\_length", "sepal\_width", "petal\_length", and "petal\_width" in the 'iris' dataframe.

The resulting plot will show scatter plots of each variable against every other variable, with histograms of each variable on the diagonal. This will allow you to quickly identify any correlations or patterns between the variables in the 'iris' dataframe.



# Annotating and formatting plots

## PROMPT STRUCTURE

I want you to act as a data scientist coding in R. Given a dataframe {dataframe name} containing the columns {column names}, use ggplot2 to plot a {chart type} the relationship between {variables}. {Insert annotation and formatting requirements}



I want you to act as a data scientist coding in R. Given a dataframe 'iris' containing the columns 'sepal\_length', 'sepal\_width', 'flower\_type', use ggplot2 to plot a scatterplot the relationship between sepal\_length and sepal\_width. Use colors to denote the different flower\_types. Name the chart "Length against Width"



As a data scientist coding in R, here's how you can use ggplot2 to create a scatterplot that shows the relationship between "sepal\_length" and "sepal\_width" in the 'iris' dataframe, with colors denoting the different "flower\_type"s, and named "Length against Width":



R

Copy code

```
# Load the ggplot2 package
library(ggplot2)

# Create the scatterplot
ggplot(data = iris, aes(x = sepal_length, y = sepal_width, color = flower_ty
geom_point() +
  labs(title = "Length against Width")
```

In this code, we first load the `ggplot2` package. We then use `ggplot()` to create the plot, specifying the 'iris' dataframe as the data source, and setting the x-axis to "sepal\_length" and the y-axis to "sepal\_width". We also use `color = flower_type` to color-code the points based on the "flower\_type" column.

We then add a `geom_point()` layer to create the scatterplot, and use `labs()` to add a title to the plot, naming it "Length against Width".

The resulting plot will show a scatterplot of "sepal\_length" against "sepal\_width", with points color-coded based on the "flower\_type" column, and a title of "Length against Width". This will allow you to quickly identify any patterns or correlations between "sepal\_length" and "sepal\_width" for different types of flowers in the 'iris' dataframe.



# Changing plot themes in ggplot2

## PROMPT STRUCTURE

I want you to act as a data scientist coding in R. Given a dataframe {dataframe name} containing the columns {column names}, use ggplot2 to plot a {chart type} the relationship between {variables}. Change the color theme to match that of {theme}



I want you to act as a data scientist coding in R. Given a dataframe 'iris' containing the columns 'sepal\_length', 'sepal\_width', 'flower\_type', use ggplot2 to plot a scatterplot the relationship between sepal\_length and sepal\_width. Change the color theme to match that of fivethirtyeight.



Sure! Here's an example code to create a scatterplot of sepal\_length vs sepal\_width using ggplot2 in R and applying the fivethirtyeight theme:

```
R Copy code

library(ggplot2)
library(hrbrthemes) # for fivethirtyeight theme

ggplot(iris, aes(x = sepal_length, y = sepal_width, color = flower_type)) +
  geom_point(size = 3) +
  scale_color_manual(values = c("#EF4F91", "#4F628E", "#FFA600")) +
  labs(title = "Sepal Length vs Sepal Width",
       x = "Sepal Length",
       y = "Sepal Width",
       color = "Flower Type") +
  theme_fivethirtyeight()
```



# Creating plots with matplotlib

## PROMPT STRUCTURE

I want you to act as a data scientist coding in Python. Given a dataframe {dataframe name} containing the columns {column names} Use matplotlib to plot a {chart type and requirement}.



I want you to act as a data scientist coding in Python. Given a dataframe 'gapminder' containing the column 'country', 'gdp', 'year'. Use matplotlib to plot a line chart of GDP against the year for each country.



Sure, here's an example code to plot a line chart of GDP against the year for each country using matplotlib:  

python

 Copy code

```
import pandas as pd
import matplotlib.pyplot as plt

# load the data
gapminder = pd.read_csv('gapminder.csv')

# create a list of unique countries
countries = gapminder['country'].unique()

# create subplots for each country
fig, axes = plt.subplots(nrows=len(countries), ncols=1, figsize=(8, 50))

# iterate through each country and plot the line chart
for i, country in enumerate(countries):
    # subset the data for the current country
    country_data = gapminder.loc[gapminder['country'] == country]

    # plot the line chart
    axes[i].plot(country_data['year'], country_data['gdp'])
    axes[i].set_title(country)
    axes[i].set_xlabel('Year')
    axes[i].set_ylabel('GDP')

# adjust the layout and spacing of the subplots
plt.tight_layout()
```



# Crating pairplots with matplotlib

## PROMPT STRUCTURE

I want you to act as a data scientist coding in Python. Given a dataframe {dataframe name} containing the columns {column names}. Use matplotlib to plot a pair plot that shows the relationship of one variable against another.



I want you to act as a data scientist coding in Python. Given a dataframe 'iris' containing the columns 'sepal\_length', 'sepal\_width', 'petal\_width', 'petal\_length'. Use matplotlib to plot a pair plot that shows the relationship of one variable against another.



Sure, here's an example code to create a pair plot using `seaborn` library:



python

Copy code

```
import seaborn as sns
import matplotlib.pyplot as plt

iris = sns.load_dataset("iris")
sns.pairplot(iris, vars=['sepal_length', 'sepal_width', 'petal_length', 'pet
plt.show()
```

This will create a 4x4 grid of scatterplots, where the diagonal shows the distribution of each variable and the off-diagonal plots show the relationship between each pair of variables.



# Annotating and formatting plots in matplotlib

## PROMPT STRUCTURE

I want you to act as a data scientist coding in Python. Given a dataframe {dataframe name} containing the columns {column names}, use matplotlib to to plot a {chart type} the relationship between {variables}. {Insert annotation and formatting requirements}



I want you to act as a data scientist coding in Python. Given a dataframe 'iris' containing the columns 'sepal\_length', 'sepal\_width', 'flower\_type', use matplotlib to plot a scatterplot the relationship between sepal\_length and sepal\_width. Use colors to denote the different flower\_types. Name the chart "Length against Width"



Sure, here's the Python code to create a scatter plot using matplotlib with the given requirements:

python

Copy code

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the iris dataset
iris = pd.read_csv('iris.csv')

# Define color palette for each flower type
colors = {'setosa': 'red', 'versicolor': 'green', 'virginica': 'blue'}

# Create a scatter plot with sepal_length on x-axis and sepal_width on y-axis
fig, ax = plt.subplots()
for flower_type in iris['flower_type'].unique():
    x = iris.loc[iris['flower_type'] == flower_type, 'sepal_length']
    y = iris.loc[iris['flower_type'] == flower_type, 'sepal_width']
    ax.scatter(x, y, c=colors[flower_type], label=flower_type, alpha=0.5)

# Set x-axis and y-axis labels
ax.set_xlabel('Sepal Length')
ax.set_ylabel('Sepal Width')

# Set plot title and legend
ax.set_title('Length against Width')
ax.legend()

# Display the plot
plt.show()
```

Regenerate response



# Changing plot themes in matplotlib

## PROMPT STRUCTURE

I want you to act as a data scientist coding in Python. Given a dataframe {dataframe name} containing the columns {column names}, use matplotlib to plot a {chart type} the relationship between {variables}. Change the color theme to match that of {theme}



I want you to act as a data scientist coding in python. Given a dataframe 'iris' containing the columns 'sepal\_length', 'sepal\_width', 'flower\_type', use matplotlib to plot a scatterplot the relationship between sepal\_length and sepal\_width. Change the color theme to match that of fivethirtyeight.



Sure, here's an example code to plot a scatterplot using matplotlib in Python:

```
python
import matplotlib.pyplot as plt
import pandas as pd

# Load the iris dataset
iris = pd.read_csv("iris.csv")

# Set the color theme to match fivethirtyeight
plt.style.use('fivethirtyeight')

# Plot the scatterplot
plt.scatter(iris['sepal_length'], iris['sepal_width'])

# Set the labels for the plot
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('Scatterplot of Sepal Length and Width')

# Show the plot
plt.show()
```

# Prompts for machine learning workflows



# Feature engineering ideation

## PROMPT STRUCTURE

I want you to act as a data scientist. Given a dataset of {dataset name} that contains the {columns}, you are to predict {predicted variable}. Suggest data that will be helpful for this problem and perform feature engineering for this problem.



I want you to act as a data scientist. Given a dataset of movies that contains the cast, year of release, their budget, and other movie data, you are to predict what is the global box office of the movie. Suggest data that will be helpful for this problem and perform feature engineering for this problem. 



To predict the global box office of a movie, we need to consider various features such as the cast, year of release, budget, and other movie data. Additionally, we can consider the following data:

1. Genre - The genre of a movie can impact its success at the box office. For example, action movies tend to perform better than drama or indie films.
2. Runtime - The length of a movie can also affect its performance at the box office. Shorter movies tend to have more showings per day, increasing their chances of making more money.
3. Release date - The release date of a movie can impact its box office performance. For example, movies released during the holiday season or summer tend to have higher box office earnings.
4. Marketing - The marketing budget and strategy can also play a significant role in a movie's success at the box office. Higher marketing budgets and effective marketing strategies can attract more viewers.

Based on this, we can perform the following feature engineering:

1. Create a new feature for the movie's genre and use one-hot encoding to convert it into a categorical variable.
2. Convert the runtime into a categorical variable by grouping movies by length, for example, less than 90 minutes, 90-120 minutes, and more than 120 minutes.



# Model training workflow

## PROMPT STRUCTURE

I want you to act as a data scientist programming in Python. Given a dataset of {dataframe name} that contains the {column name}, write code to predict {output variable}.



I want you to act as a data scientist programming in Python. Given a dataset of movies that contains the cast, year of release, their budget, and other movie data, write code to predict what is the global box office of the movie.



To predict the global box office of the movie, we need to perform some preprocessing and feature engineering on the movie dataset. Here's a general outline of the steps we can take:

1. Load the movie dataset into a pandas dataframe.
2. Drop irrelevant columns that are not related to global box office, such as movie title or director.
3. Handle missing values in the data. We can use methods like mean, median or interpolation to replace missing values in numerical columns, and mode for categorical columns.
4. Extract relevant features from the data, such as release year or total cast members.
5. Convert categorical data to numerical data using techniques like one-hot encoding or label encoding.
6. Split the data into training and test sets.
7. Train a regression model on the training set, such as linear regression, decision tree regression or random forest regression.
8. Evaluate the model on the test set using metrics like mean squared error or R-squared.

Here's some example Python code that performs some of these steps:

python

Copy code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```



# Hyperparameter tuning workflow

## PROMPT STRUCTURE

I want you to act as a data scientist programming in Python.  
Given a {type of model} model, write code to tune the hyperparameter.

### Example:

The screenshot shows a chat interface with a user prompt and a GPT response. The user prompt is: "I want you to act as a data scientist programming in Python. Given a decision tree classification model, write code to tune the hyperparameter." The GPT response is: "Sure, here is an example code to tune hyperparameters of a decision tree classification model using GridSearchCV in Python:" followed by a code block. The code block is titled "python" and contains the following code:

```
python
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.datasets import load_iris

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Define decision tree classifier
dt = DecisionTreeClassifier()

# Define hyperparameter grid
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 1, 2, 3, 4, 5],
    'min_samples_split': [2, 3, 4, 5],
    'min_samples_leaf': [1, 2, 3]
}
```



# Model explainability workflow

## PROMPT STRUCTURE

I want you to act as a data scientist programming in Python. Given a {type of model} that predicts the {predictor variable}, write code that explains an output using Shap values.

### Example:



I want you to act as a data scientist programming in Python . Given a sklearn decision tree model that predicts the box office of a model based on its the cast, year of release, their budget, and other movie data, write code that explains an output using Shap values.



Sure! Here's an example code snippet that explains the output of a sklearn decision tree model using Shap values:

```
python Copy code

import shap
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeRegressor

# Load the movie dataset
movie_data = pd.read_csv('movie_data.csv')

# Separate the target variable from the features
X = movie_data.drop('box_office', axis=1)
y = movie_data['box_office']

# Train a decision tree regression model
model = DecisionTreeRegressor(max_depth=3)
model.fit(X, y)

# Generate the Shap values for a single movie
movie = X.iloc[[0]]
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(movie)
```



# Model training workflow

## PROMPT STRUCTURE

I want you to act as a data scientist programming in R.  
Given a dataframe of {dataframe name} that contains  
{column names}, write code to predict {output}.

### Example: Train a classification model in R



I want you to act as a data scientist programming in R. Given a dataframe of products that contains product\_names, price and product\_category, write code to predict the product\_category of a new product.



Sure, here's an example of how you can use a decision tree classifier from the `rpart` package in R to predict the product\_category of a new product based on the product\_names and price.

```
r Copy code

# load necessary packages
library(rpart)
library(dplyr)

# read in the data
products <- read.csv("products.csv")

# perform feature engineering if necessary
# ...

# split the data into training and testing sets
set.seed(123)
train_index <- sample(nrow(products), 0.8 * nrow(products))
train_data <- products[train_index, ]
test_data <- products[-train_index, ]

# train a decision tree classifier on the training data
tree_model <- rpart(product_category ~ product_names + price, data = train_data)

# make predictions on the testing data
predictions <- predict(tree_model, newdata = test_data)
```



# Hyperparameter tuning workflow

## PROMPT STRUCTURE

I want you to act as a data scientist programming in R.  
Given a {type of model} model, write code to tune the hyperparameter.

### Example:



I want you to act as a data scientist programming in R. Given a decision tree classification model, write code to tune the hyperparameter.



To tune the hyperparameters of a decision tree classification model in R, we can use the ``rpart`` package which provides the ``rpart.control()`` function to specify the tuning parameters. Here's an example code:

R

Copy code

```
library(rpart)

# Load the dataset
data <- read.csv("products.csv")

# Split the data into training and testing sets
set.seed(123)
train_idx <- sample(nrow(data), 0.7*nrow(data))
train_data <- data[train_idx,]
test_data <- data[-train_idx,]

# Define the formula for the model
formula <- product_category ~ product_names + price

# Define the rpart control parameters for tuning
ctrl <- rpart.control(minsplit = 5, maxdepth = 5, cp = 0.01)

# Build the decision tree model
tree_model <- rpart(formula, data = train_data, method = "class", control =

# Predict on the testing set
predictions <- predict(tree_model, test_data, type = "class")
```



# Model explainability workflow

## PROMPT STRUCTURE

I want you to act as a data scientist programming in R .  
Given a {type of model} that predicts the {predictor variable},  
write code that explains an output using Shap values.

### Example:



I want you to act as a data scientist programming in R. Given a decision tree model that predict the product\_category of a product based on product\_name and price, write code that explains an output using Shap values.



Sure! Here's an example code snippet that explains the output of a decision tree model using SHAP values in R:



scss

Copy code

```
# Load required packages
library(randomForest)
library(data.table)
library(shap)

# Load data
data(iris)

# Train decision tree model
model <- randomForest(Species ~ ., data = iris)

# Convert data to data.table format
iris_dt <- as.data.table(iris)

# Generate SHAP values
shap_values <- shapley(model, iris_dt[, -5])

# Extract SHAP values for a single observation
obs_shap_values <- shap_values[[1]]

# Plot SHAP values for the observation
plot(obs_shap_values, iris_dt[1, -5])
```

# Prompts for time series analysis workflows



# Changing time horizons using pandas

## PROMPT STRUCTURE

I want you to act as a data scientist coding in Python. Given a time series data in a Pandas dataframe {dataframe name} with timestamp Index in {original frequency} frequency with one column {column name}, convert the timestamp frequency to {desired frequency}.



I want you to act as a data scientist coding in Python. Given a time series data in a Pandas dataframe `ts` with timestamp Index in daily frequency with one column `value`, convert the timestamp frequency to weekly and monthly.



Sure, here's some sample code to convert the daily frequency of a time series data in a Pandas dataframe to weekly and monthly:

python Copy code

```
import pandas as pd

# Example time series data with daily frequency
dates = pd.date_range(start='2022-01-01', end='2022-02-28', freq='D')
ts = pd.DataFrame({'value': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,

# Convert daily frequency to weekly frequency
ts_weekly = ts.resample('W').sum()

# Convert daily frequency to monthly frequency
ts_monthly = ts.resample('M').sum()
```



## Build test series model

### PROMPT STRUCTURE

I want you to act as a data scientist coding in Python. Given a time series data in a dataframe {dataframe name} with timestamp Index in {original frequency} frequency with one column {column name}, build a forecasting model, assuming data is stationary.



I want you to act as a data scientist coding in Python. Given a time series data in a Pandas dataframe `ts` with timestamp Index in daily frequency with one column `value`, build a forecasting model assuming that the time series is stationary.



Sure, here's an example of how to build a forecasting model for a stationary time series in Python using the ARIMA model:

python

Copy code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.arima_model import ARIMA

# Load the data
ts = pd.read_csv('data.csv', index_col='timestamp', parse_dates=True)

# Ensure that the data is stationary
# e.g., by differencing the data
ts_diff = ts.diff().dropna()

# Split the data into training and testing sets
train_size = int(len(ts_diff) * 0.8)
train_data, test_data = ts_diff[0:train_size], ts_diff[train_size:]

# Build the ARIMA model
# p=1, d=1, q=1 corresponds to an ARIMA(1,1,1) model
model = ARIMA(train_data, order=(1, 1, 1))
model_fit = model.fit(dispatch=0)

# Generate predictions on the test data
predictions = model_fit.predict(start=len(train_data), end=len(ts_diff)-1, d
```



# Perform stationarity test

## PROMPT STRUCTURE

I want you to act as a data scientist coding in Python. Given a time series data in a dataframe {dataframe name} with timestamp Index in {original frequency} frequency with one column {column name}, perform a Dicky Fuller test.



I want you to act as a data scientist coding in Python. Given a time series data in a Pandas dataframe `ts` with timestamp Index in daily frequency with one column `value`, perform a Dicky Fuller test.



Sure, here's an example code to perform a Dickey-Fuller test on a time series data in a Pandas dataframe `ts` with timestamp Index in daily frequency with one column `value`:

python

Copy code

```
from statsmodels.tsa.stattools import adfuller

# Run Dickey-Fuller test on time series data
result = adfuller(ts['value'])

# Print test results
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

The `adfuller()` function from the `statsmodels` library is used to perform the Dickey-Fuller test on the time series data in the `ts` dataframe. The test results are printed, including the ADF statistic, p-value, and critical values. The ADF statistic is used to determine whether the time series is stationary or not. If the p-value is less than the significance level (e.g. 0.05), we reject the null hypothesis of non-stationarity and conclude that the time series is stationary.



# Changing time horizons

## PROMPT STRUCTURE

I want you to act as a data scientist coding in R. Given a time series data in a dataframe {dataframe name} with timestamp Index in {original frequency} frequency with one column {column name}, convert the timestamp frequency to {desired frequency}



I want you to act as a data scientist coding in R. Given a time series data in a dataframe `ts` with timestamp Index in daily frequency with one column `value`, convert the timestamp frequency to weekly and monthly.



As a data scientist coding in R, here's how you can convert a time series data in a dataframe `ts` with timestamp index in daily frequency to weekly and monthly:

R

Copy code

```
# Load the lubridate package for working with time data
library(lubridate)

# Convert daily frequency to weekly frequency
ts_weekly <- ts %>%
  as.data.frame() %>%
  mutate(date = as.Date(index(ts))) %>%
  group_by(year = year(date), week = week(date)) %>%
  summarise(value = mean(value)) %>%
  ungroup() %>%
  mutate(date = as.Date(paste0(year, "-W", week, "-1"))) %>%
  select(-year, -week) %>%
  column_to_rownames("date") %>%
  ts(frequency = "52")

# Convert daily frequency to monthly frequency
ts_monthly <- ts %>%
  as.data.frame() %>%
  mutate(date = as.Date(index(ts))) %>%
  group_by(year = year(date), month = month(date)) %>%
  summarise(value = mean(value)) %>%
  ungroup() %>%
  mutate(date = as.Date(paste0(year, "-", month, "-1"))) %>%
  select(-year, -month) %>%
  column_to_rownames("date") %>%
  ts(frequency = "12")
```



## Build test series model

### PROMPT STRUCTURE

I want you to act as a data scientist coding in R. Given a time series data in a dataframe {dataframe name} with timestamp Index in {original frequency} frequency with one column {column name}, build a forecasting model, assuming data is stationary.



I want you to act as a data scientist coding in R. Given a time series data in a dataframe `ts` with timestamp Index in daily frequency with one column `value`, build a forecasting model, assuming data is stationary.



Sure, I can help you with that. Here's an example code in R to build a forecasting model for a stationary time series using the ARIMA model:

R

Copy code

```
# Load required libraries
library(forecast)

# Load data
ts <- read.csv("your_file_path.csv", header=TRUE, sep=",")

# Convert 'ts' to time series object
ts <- ts$value
ts <- ts(ts, frequency = 365)

# Split data into training and test sets
train <- window(ts, end = c(2022,12,31))
test <- window(ts, start = c(2023,1,1))

# Fit an ARIMA model to the training data
fit <- auto.arima(train)

# Make forecasts for the test set
forecast <- forecast(fit, h = length(test))

# Print the forecasted values and accuracy measures
print(forecast)
accuracy(forecast, test)
```



# Perform stationarity test

## PROMPT STRUCTURE

I want you to act as a data scientist coding in R. Given a time series data in a dataframe {dataframe name} with timestamp Index in {original frequency} frequency with one column {column name}, perform a Dicky Fuller test.



I want you to act as a data scientist coding in R. Given a time series data in a dataframe `ts` with timestamp Index in daily frequency with one column `value`, perform a stationarity test.



Sure, I can help you with that. One popular stationarity test is the Augmented Dickey-Fuller (ADF) test. Here's an example code in R to perform the ADF test on your time series data:

R

Copy code

```
# Load required libraries
library(tseries)

# Load data
ts <- read.csv("your_file_path.csv", header=TRUE, sep=",")

# Convert 'ts' to time series object
ts <- ts$Value
ts <- ts(ts, frequency = 365)

# Perform ADF test
result <- adf.test(ts)
```

In the code above, we first load the "tseries" library, which contains the `adf.test` function that we will use for the ADF test. Then, we load the time series data from a CSV file (replace "your\_file\_path.csv" with the actual path to your data file), and convert it to a time series object using the `ts` function. Note that we set the frequency of the time series to 365, assuming that the data is in daily frequency.

Finally, we perform the ADF test using the `adf.test` function, which returns a list containing the test statistics, p-value, and other information. You can access the test result by examining the `result$statistic` and `result$p.value` values.

# Prompts for natural language processing workflows



# Classify text sentiment

## PROMPT STRUCTURE

I want you to act as a sentiment classifier. Classify the following text which came from {describe text origin} as “positive”, “negative”, “neutral” or “unsure”: {Insert text to be classifier}



I want you to act as a sentiment classifier. Classify the following reviews on a self-help book as “positive”, “negative”, “neutral” or “unsure”:

Amazing read



I dozed off.

Chef’s kiss.



As a sentiment classifier, I would classify the following reviews on a self-help book as follows:

1. "Amazing read" - Positive
2. "😞" - Unsure (as it's an emoticon that can represent different emotions such as sadness or joy depending on the context)
3. "I dozed off." - Negative
4. "Chef's kiss." - Positive

Note that the second review containing an emoticon can be difficult to classify without additional context. If this review was associated with positive or negative sentiment in the context of the book or the review as a whole, we could classify it as positive or negative accordingly. However, as it stands alone without additional context, we would be unsure of its sentiment.



# Create regular expressions

## PROMPT STRUCTURE

I want you to act as a programmer coding in Python, use regular expressions to test if a string {insert requirements}



I want you to act as a programmer coding in Python, use regular expressions to test if a string starts with a number, and ends with a "!"



As a programmer coding in Python, here's how you can use regular expressions to test if a string starts with a number and ends with a "!"



python

Copy code

```
import re

string = "7 Hello world!"

# regular expression pattern
pattern = r"^\d.*!"

# check if the string matches the pattern
if re.match(pattern, string):
    print("The string starts with a number and ends with a '!")
else:
    print("The string does not match the pattern")
```

In this code, we first import the `re` module to use regular expressions in Python. We then define our string and regular expression pattern. The pattern `^\d.*!` means that the string should start with a digit (`\d`), followed by any number of characters (`.*`), and end with a "!" (`!`). The `^` and `$` symbols indicate that the pattern should match the entire string, from start to end.

We then use the `re.match()` function to check if the string matches the pattern. If it does, we print a message saying that the string starts with a number and ends with a "!", otherwise, we print a message saying that the string does not match the pattern.



# Text dataset generation

## PROMPT STRUCTURE

I want you to act as a dataset generator. Please generate {number of text} texts on {required text and the context}. {Insert additional requirements}



I want you to act as a dataset generator. Please generate 5 text for car review on a second-hand car dealership. Please include data that are positive, neutral, and negative in sentiments.



As a dataset generator, here are five text reviews for a second-hand car dealership, including positive, neutral, and negative sentiments:

1. Positive: "I couldn't be happier with my purchase from this dealership! The staff was incredibly helpful and patient with me as I made my decision, and the car is in excellent condition. Highly recommend!"
2. Neutral: "Overall, my experience at this dealership was fine. The car I was interested in was available and I was able to take it for a test drive. However, the staff wasn't particularly friendly or welcoming, and I didn't feel like I received much guidance during the process."
3. Negative: "I had a terrible experience at this dealership. The car I was interested in wasn't even available to test drive, and the salesperson was incredibly pushy and aggressive. I felt like I was being pressured into making a purchase I wasn't comfortable with. I would not recommend this dealership."
4. Positive: "The staff at this dealership went above and beyond to help me find the perfect car for my needs. They were knowledgeable and professional, and made the entire process easy and stress-free. I'm so grateful for their help!"
5. Negative: "I purchased a car from this dealership and quickly realized that it was in much worse condition than I had been led to believe. The staff was unhelpful and unresponsive when I tried to address the issues, and I ended up spending thousands of dollars on repairs. I would never recommend this dealership to anyone."



# Machine translation

## PROMPT STRUCTURE

I want you to act as a translator. Please translate {phrase} from {origin language} to {translated language}.



I want you to act as a translator. Can you please explain what machine learning is in french?



Bien sûr, je peux vous expliquer ce qu'est l'apprentissage automatique en français.



L'apprentissage automatique, également connu sous le nom de "machine learning" en anglais, est un domaine de l'informatique qui utilise des algorithmes pour permettre à un ordinateur d'apprendre à partir de données et de prendre des décisions ou de prédire des résultats sans être explicitement programmé pour cela.

En utilisant des modèles mathématiques et statistiques, l'apprentissage automatique peut analyser des données complexes, détecter des schémas et des tendances, et utiliser ces informations pour résoudre des problèmes ou prendre des décisions.

Les exemples courants d'applications de l'apprentissage automatique comprennent la reconnaissance vocale, la recommandation de produits, la détection de fraude, la prédiction de l'évolution des marchés financiers, la reconnaissance d'images, et bien d'autres encore.

# Conceptual and career oriented prompts



## Explain data concepts for business executives

### PROMPT STRUCTURE

I want you to act as a data scientist of a corporate company. {Describe content in detail, if required} Please explain to a business executive what {concept} means.



I want you to act as a data scientist of a corporate company. You have performed an AB test on a new font on an ecommerce website, and the results are that the null hypothesis that "the change in font has no effect on conversion rate" is rejected. Please explain to a business executive what this means.



As a data scientist of our corporate company, I am happy to explain the results of our recent A/B test on the new font for our ecommerce website. 

We set up the A/B test with the hypothesis that changing the font would have no effect on the conversion rate. After running the experiment, we analyzed the results and found that the null hypothesis was rejected, meaning that the change in font did have a statistically significant effect on the conversion rate.

This means that the new font we tested had an impact on our website's performance, either positive or negative. We can now conclude that the change in font was not insignificant and that it did have a significant impact on the conversion rate.

The next steps would be to analyze the data and try to understand the reasons behind the change in conversion rate. We should also consider the cost and benefits of implementing the new font on our website. Based on our findings, we can decide whether to continue using the new font or revert to the old one.

Overall, the results of this A/B test provide us with valuable insights and help us make data-driven decisions for our ecommerce website.



## Summarize article/paper

### PROMPT STRUCTURE

I want you to act as a data scientist in a research start-up. Please explain the paper {paper} to a {level of difficulty, e.g. software developer, five-year-old, business executive, professor}.



I want you to act as a data scientist in a research start-up. Please explain the paper "Attention is all you need" to a software developer.



As a data scientist in a research start-up, I would be happy to explain the paper "Attention is all you need" to a software developer.

"Attention is all you need" is a paper that was published in 2017 by researchers at Google. The paper introduces a new type of neural network architecture called the Transformer, which is based solely on attention mechanisms and doesn't use any recurrent or convolutional layers.

The paper explains how the Transformer can be used for tasks such as machine translation, language modeling, and image captioning. The Transformer architecture allows for more efficient training and faster inference than previous models, while achieving state-of-the-art results on several benchmarks.

The main idea behind the Transformer is the concept of "self-attention". Self-attention allows the model to weigh the importance of different parts of the input sequence when making predictions. This is especially useful for tasks involving sequences of varying length, such as natural language processing.

The paper also introduces the concept of multi-head attention, which allows the model to attend to different positions in the input sequence simultaneously. This improves the model's ability to capture complex patterns in the data.

Overall, "Attention is all you need" is an important paper in the field of deep learning, as it introduces a new type of architecture that has since been used in many state-of-the-art models. Its concepts have become increasingly important in natural language processing, where the Transformer architecture has had a significant impact.



## Suggest portfolio projects and ideas

### PROMPT STRUCTURE

I want you to act as a data science career coach. I am a {describe your background} and I would like to {describe career objective}. Suggest portfolio projects and ideas {describe objective of portfolio}



I want you to act as a data science career coach. I am a final year student in Chemical Engineering and I would like to pivot into data science. Suggest portfolio projects and ideas that will show my capability in time series forecasting in a chemical engineering domain.



As a final year student in Chemical Engineering looking to pivot into data science, it's great that you are interested in time series forecasting in a chemical engineering domain. Here are some portfolio project ideas that will showcase your capabilities in this area:

1. Predictive maintenance for chemical plants: Use time series data from sensors and other equipment in a chemical plant to build a model that predicts equipment failure and maintenance needs. This can help optimize maintenance schedules, reduce downtime, and improve safety.
2. Forecasting energy consumption for a chemical plant: Use historical energy consumption data to build a model that predicts future energy usage. This can help identify areas where energy can be saved, and optimize energy usage across the plant.
3. Quality control for chemical processes: Use time series data from sensors and other equipment to build a model that predicts the quality of the final product. This can help identify problems early in the production process and reduce waste.
4. Sales forecasting for chemical products: Use historical sales data to build a model that predicts future sales. This can help with inventory management, production planning, and overall business strategy.
5. Forecasting chemical reactions: Use time series data from chemical reactions to build a model that predicts the outcome of the reaction. This can help optimize reaction conditions and reduce waste.

When working on these projects, be sure to document your process and methodology, including data cleaning and preprocessing, feature engineering, model selection, and evaluation. Good luck with your portfolio!



## Write tutorials

### PROMPT STRUCTURE

I want you to act as a data scientist writer. Please write the {number-of-words}-word introduction to a tutorial on {title}. {Insert relevant key points}.



I want you to act as a data scientist writer. Please write the 100-word introduction to a tutorial on "Learning to Analyze Dirty Data in Pandas"



In the world of data science, cleaning and preparing data is often the most time-consuming and challenging aspect of any project. Real-world data is rarely in the format we need it to be, and it often contains errors, missing values, and inconsistencies. In this tutorial, we will explore how to use Pandas, a powerful Python library for data analysis, to clean and prepare dirty data. We will cover common data cleaning techniques, such as handling missing data, removing duplicates, and dealing with outliers. By the end of this tutorial, you will have the skills to turn messy data into clean, usable data that you can analyze with confidence.



# About this cheat sheet



Author:

**Travis Tang**

 [LinkedIn](#)

 [Github](#)

References:

[60 ChatGPT prompts for data science](#)

[ChatGPT cheat sheet](#)